Architecture multi-agents de DressMe

L'orchestrateur et le protocole A2A

DressMe n'est pas un seul assistant : c'est une équipe d'agents spécialisés. Chaque agent sait faire une chose (conseiller une tenue, lire la garde-robe, planifier une valise, trouver ce qui manque). Deux mécanismes les font travailler ensemble : un **orchestrateur** qui choisit le bon agent pour chaque demande, et le protocole **A2A** (Agent2Agent) qui permet à un agent d'en appeler un autre à travers le réseau.

Principe commun : **le LLM décide et rédige, le code déterministe vérifie et calcule**. Aucun agent ne peut citer une pièce absente de la garde-robe, inventer la météo ou inventer une source.

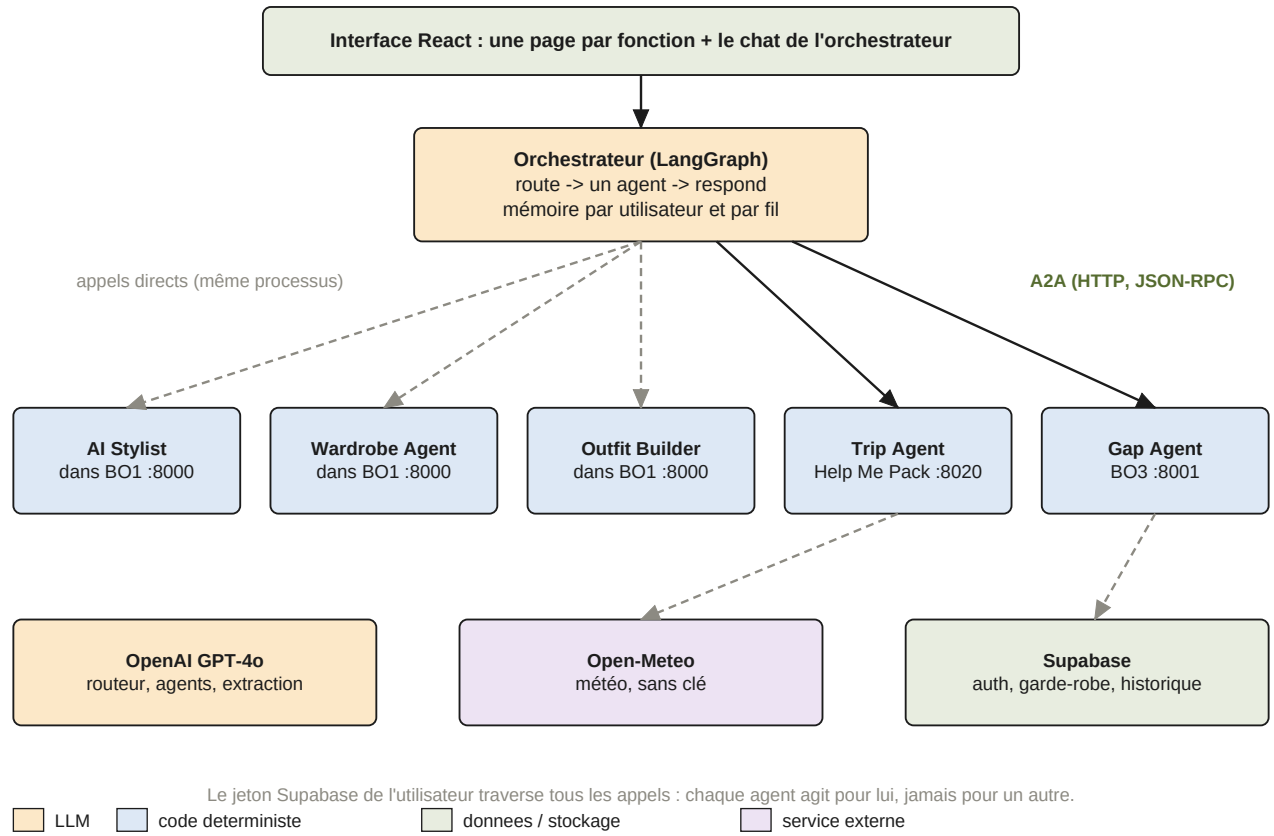

Figure 1 - vue d'ensemble : l'orchestrateur reçoit la demande, un agent répond.

Pourquoi plusieurs agents plutôt qu'un seul ?

- **Spécialisation** : chaque agent a ses outils et son prompt courts, donc moins d'erreurs qu'un seul agent à tout faire.
- **Indépendance** : le Trip Agent (Help Me Pack) et le Gap Agent sont des services séparés, avec leur propre port, leurs propres tests et leur propre équipe ; on peut en redémarrer un sans toucher aux autres.
- **Réutilisation** : le Trip Agent demande la garde-robe au Wardrobe Agent au lieu de relire la base lui-même ; une seule règle d'accès aux données.
- **Sécurité** : chaque appel porte le jeton de l'utilisateur ; chaque agent ne voit que ses données.

Les agents

Agent	Où	Rôle	Joint par
AI Stylist	BO1 :8000	Conseils de tenue avec météo et livres de mode (RAG)	appel direct, A2A
Wardrobe Agent	BO1 :8000	Chercher, compter, comparer, corriger une pièce (une correction demande une confirmation)	appel direct, A2A
Outfit Builder	BO1 :8000	Trois tenues complètes pour une occasion et une saison (LangGraph)	appel direct, A2A
Similar Agent	BO1 :8000	Quelles pièces possédées ressemblent à celles d'une photo	A2A
Trip Agent	Help Me Pack :8020	Météo, plus petit ensemble de vêtements qui habille chaque jour (OR-Tools CP-SAT), checklist	A2A
Gap Agent	BO3 :8001	Classe les pièces qui manquent pour un objectif (XGBoost)	A2A

Les trois premiers vivent dans le même processus que l'orchestrateur de BO1 : il les appelle directement. Le Trip Agent et le Gap Agent sont d'autres services : il les appelle en A2A, exactement comme n'importe quel autre client A2A le ferait. Chaque agent A2A publie le même type de carte et répond au même format.

L'orchestrateur (BO1, LangGraph)

Un seul point d'entrée de conversation (`POST /api/assistant/chat`). Le graphe a un nœud de routage, un nœud par agent, puis un nœud qui répond.

Nœud	Ce qu'il fait
route	choisit l'agent : le LLM répond en JSON et n'a le droit de citer que les agents de la liste ; des règles FR/EN (valise, voyage, manque, tenue...) servent de filet quand le LLM est absent ou se trompe
stylist, wardrobe, outfit	appel direct dans le même processus
trip, gap	appel A2A avec le jeton de l'utilisateur ; si le service est arrêté, message clair et pas d'erreur 500
redirect	« Should I buy » et « Find Similar » ont besoin d'une photo : la réponse ouvre la bonne page
clarify	la demande est trop vague ou c'est une salutation : demande ce que l'utilisateur veut
respond	écrit le tour dans l'historique (8 derniers échanges)

La mémoire d'une conversation

Le graphe est compilé avec un point de reprise (checkpointer) : un fil par utilisateur et par conversation, deux utilisateurs ne partagent jamais un fil. Exemple :

```
Utilisateur : « Prépare ma valise pour Paris » -> Trip Agent : tâche input-required, « quelles
dates ? »
Utilisateur : « du 3 au 7 juin, 8 kg » -> aucun mot-clé de voyage, mais la question est restée
ouverte : la réponse repart au Trip Agent avec le même contextId.
```

- Une correction proposée par le Wardrobe Agent n'est continuée que par un oui ou un non ; toute autre phrase est une nouvelle demande.
- L'interface indique l'agent qui a répondu et ouvre la page voulue par un bouton (par exemple Help Me Pack, sur la même session de voyage).

Le protocole A2A (Agent2Agent)

A2A est un protocole ouvert qui permet à des agents de se découvrir et de se confier des tâches, sans connaître le code les uns des autres. DressMe implémente la version 0.3 au-dessus de HTTP et JSON-RPC 2.0 (dossier `a2a_protocol/`) : un serveur (`server.py`) et un client (`client.py`).

Figure 2 - un échange A2A, de la découverte à la réponse.

Les quatre notions à retenir

Notion	Définition	Dans DressMe
Agent Card	La carte d'identité de l'agent, publique : nom, version, compétences, mode d'authentification	`/.well-known/agent-card.json` sous chaque agent
Skill	Une compétence annoncée sur la carte, avec exemples	*plan-trip-packing*, *wardrobe-gaps*, *wardrobe-search*, *duplicate-check*...
Task	Une demande en cours, avec un état : completed, input-required, failed, canceled	*input-required* = l'agent pose une question (dates manquantes, confirmation)
contextId	Identifiant de la conversation : le rappeler continue la même tâche	le Trip Agent y met l'identifiant de sa session de voyage

Les méthodes

Méthode JSON-RPC	Rôle
`message/send`	envoie du texte et/ou des données structurées, reçoit la tâche et son résultat
`tasks/get`	relit une tâche (chaque tâche appartient à celui qui l'a créée)
`tasks/cancel`	annule une tâche

Non implémentés, et déclarés comme tels sur la carte : le streaming et les notifications push.

Qui parle à qui, par A2A

Appelant	Appelé	Pour quoi faire
Orchestrateur	Trip Agent (8020, /a2a/trip)	planifier une valise
Orchestrateur	Gap Agent (8001, /a2a/gap)	ce qui manque dans la garde-robe
Trip Agent	Wardrobe Agent (8000, /a2a/wardrobe)	lire la garde-robe de l'utilisateur (repli sur la route BO1 si A2A échoue)
Trip Agent	Gap Agent (8001)	bouton « Gap analysis » de Help Me Pack
Gap Agent	Wardrobe Agent	lire la garde-robe (repli sur Supabase)

Pour voir les cartes de tous les agents : `python scripts/a2a_discover.py`, et avec un jeton `A2A_TOKEN=... python scripts/a2a_discover.py --ask` pour envoyer une tâche à chacun.

Sécurité, pannes et limites

Authentification

- Chaque appel A2A porte l'en-tête `Authorization: Bearer` avec le jeton Supabase de l'utilisateur ; le serveur de l'agent le vérifie. Sans jeton valide : « Please sign in again ».
- Seule la carte est publique. Une tâche n'est relisible que par son propriétaire.
- Le jeton circule dans la configuration du tour, jamais dans l'état du graphe ni dans les traces.

Quand un agent est en panne

- Le client A2A a un délai maximum par appel (30 s pour la garde-robe, 90 s pour le Gap Agent, 120 s depuis l'orchestrateur).
- Orchestrateur : « The Trip Agent is not reachable right now... », jamais une erreur 500.
- Trip Agent : si la garde-robe n'est pas joignable en A2A, il retombe sur la route normale de BO1.

Ce qu'il faut dire clairement

- Chaque saut A2A ajoute un appel réseau, et souvent un appel LLM : plus précis, mais plus lent et plus cher qu'un filtrage direct.
- Le routage de l'orchestrateur repose sur le LLM, avec des règles FR/EN en secours : une demande ambiguë peut tomber sur « clarify ».
- Les tâches A2A sont gardées en mémoire (bornées) : elles disparaissent au redémarrage du service.
- Le streaming n'est pas géré : l'utilisateur attend la réponse complète.
- Il existe aussi une pile séparée de l'équipe, non branchée à l'interface React : le Stylist BO4 (Ollama, :8061) et son propre orchestrateur (:8070, routage par services.yaml vers stylist, garde-robe, achat, RAG). Il ne parle pas A2A : il appelle ces services par leurs routes HTTP.

Fichiers à connaître

Sujet	Fichier
Orchestrateur de l'application	backend/app/orchestrator_agent.py
Serveur et client A2A	a2a_protocol/server.py, a2a_protocol/client.py
Agents A2A de BO1	backend/app/a2a_agents.py, backend/app/wardrobe_a2a.py
Trip Agent en A2A	backend/main.py (handle_trip, /a2a/trip)
Gap Agent en A2A	DressMe UI Blueprint/backend/a2a_gap.py
Découverte des agents	scripts/a2a_discover.py